Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 26/01/2026**	**Enrollment No: 92301733012**

Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression

IDE: Google Colab

Theory:

Linear regression is a method for determining the best linear relationship between two variables *X* and *Y*. If variables *X* and *Y* are uncorrelated, it is pointless embarking upon linear regression. However, if a reasonable degree of correlation exists between *X* and *Y* then linear regression may be a useful means to describe the relationship between the two variables. The usual approach is to use the *least-squares* method, which minimizes the squared difference between the actual data points and a straight line. Let $[x_i, y_i]$, $i = 1,2,3,\ldots,N$ be the *N* pairs of data values of the variables *X* and *Y*. The straight-line relating *X* and *Y* is $y = mx + c$, where *m* and *c* are the gradient and constant values (to be determined) defining the straight line. Thus, $y(x_i) - y_i$ is the difference between the line and data point *i* (see Fig. 1). Taking all the data points, we seek values of *m* and *c* that minimize the squared difference *SD*.

$$\sum_{1}^{N} \left[y(x_i) - y_i \right]^2$$

This is achieved by calculating the partial derivatives of *SD* with respect to *m* and *c* and finding the pair [*m*,*c*] such that *SD* is at a minimum.

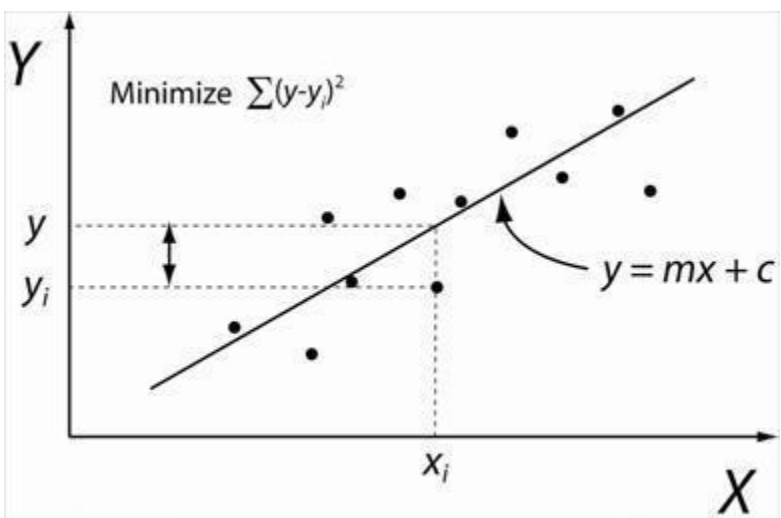


Figure 1: Illustration of Linear Regression. Linear least squares regression, the idea is to find the line y = mx + c that minimizes the mean squared difference between the line and the data points

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 26/01/2026**	**Enrollment No: 92301733012**

Batch Gradient Descent:

Gradient Descent is an optimization algorithm used for minimizing the cost function in various machine learning algorithms. It is basically used for updating the parameters of the learning model. Batch gradient descent which processes all the training examples for each iteration of gradient descent. But if the number of training examples is large, then batch gradient descent is computationally very expensive.

Let m be the number of training examples. Let n be the number of features.

Algorithm for batch gradient descent :

Let $h_\theta(x)$ be the hypothesis for linear regression. Then, the cost function is given by:
Let Σ represents the sum of all training examples from i=1 to m.
$J_{train}(\theta) = (1/2m)\ \Sigma(\ h_\theta(x^{(i)})\ -\ y^{(i)})^2$

Repeat {
$\theta j = \theta j - (\text{learning rate}/m) * \Sigma(\ h_\theta(x^{(i)})\ -\ y^{(i)})x_j^{(i)}$
For every j =0 …n
}
Where $x_j^{(i)}$ Represents the jth feature of the ith training example. So if *m* is very large(e.g. 5 million training samples), then it takes hours or even days to converge to the global minimum.That's why for large datasets, it is not recommended to use batch gradient descent as it slows down the learning.

Pre Lab Exercise:

a. Explain the meaning of linear regression

Linear regression is a statistical and machine learning technique that models the linear relationship between a dependent variable and one or more independent variables by fitting a straight line to the data.

b. Write three applications of linear regression

Three applications of linear regression are:

1. House price prediction based on features like area, location, and number of rooms
2. Sales forecasting by analyzing past sales data
3. Salary prediction based on experience, education, or skills

c. Write three advantages of linear regression

Three advantages of linear regression are:

1. Simple and easy to understand the model is straightforward and interpretable
2. Computationally efficient works well even with large datasets
3. Easy to implement and analyse requires fewer parameters and assumptions compared to complex models

d. Write three limitations of linear regression

Three limitations of linear regression are:

1. Assumes a linear relationship between variables, which may not hold for real-world data
2. Sensitive to outliers, which can significantly affect the model
3. Poor performance with complex or non-linear data patterns

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Pre-process the data
5. Visualize the Data
6. Separate the feature and prediction value columns
7. Write the Hypothesis Function
8. Write the Cost Function
9. Write the Gradient Descent optimization algorithm
10. Apply the training over the dataset to minimize the loss
11. Find the best fit line to the given dataset
12. Observe the cost function vs iterations learning curve

Program (Code):

To be attached with

1. Simple linear regression

```
import numpy as np
import matplotlib.pyplot as plt
def estimate_coef(x, y):
 # number of observations/points
 n = np.size(x)
```

```
# mean of x and y vector
 m_x = np.mean(x)
 m_y = np.mean(y)

# calculating cross-deviation and deviation about x
 SS_xy = np.sum(y*x) - n*m_y*m_x
 SS_xx = np.sum(x*x) - n*m_x*m_x

# calculating regression coefficients
 b_1 = SS_xy / SS_xx
 b_0 = m_y - b_1*m_x

 return (b_0, b_1)
def plot_regression_line(x, y, b):
# plotting the actual points as scatter plot
 plt.scatter(x, y, color = "m",
 marker = "o", s = 30)

# predicted response vector
 y_pred = b[0] + b[1]*x

# plotting the regression line
 plt.plot(x, y_pred, color = "g")

# putting labels
 plt.xlabel('x')
 plt.ylabel('y')
def main():
 # observations / data
 x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
 y = np.array([1, 3, 2, 5, 7, 8, 8, 9, 10, 12])

 # estimating coefficients
 b = estimate_coef(x, y)
 print("Estimated coefficients:\nb_0 = {} \n\tb_1 = {}".format(b[0], b[1]))

 # plotting regression line
 plot_regression_line(x, y, b)
 plt.show() # Add this to display the plot

main()
```

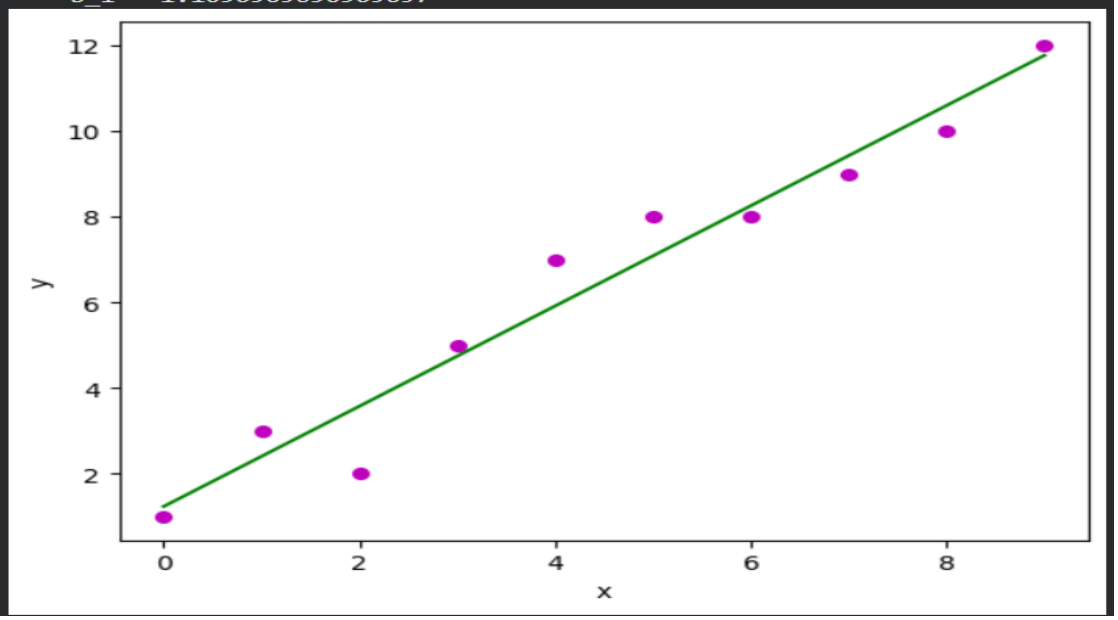
Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 26/01/2026**	**Enrollment No: 92301733012**

Results:

To be attached with

a. Datapoints scattering (without best fit line)
b. Scatter plot showing the Best fit line in the last iteration of training

```
Estimated coefficients:
b_0 = 1.2363636363636363
    b_1 = 1.1696969696969697
```



2. Multiple Linear Regression

Code:

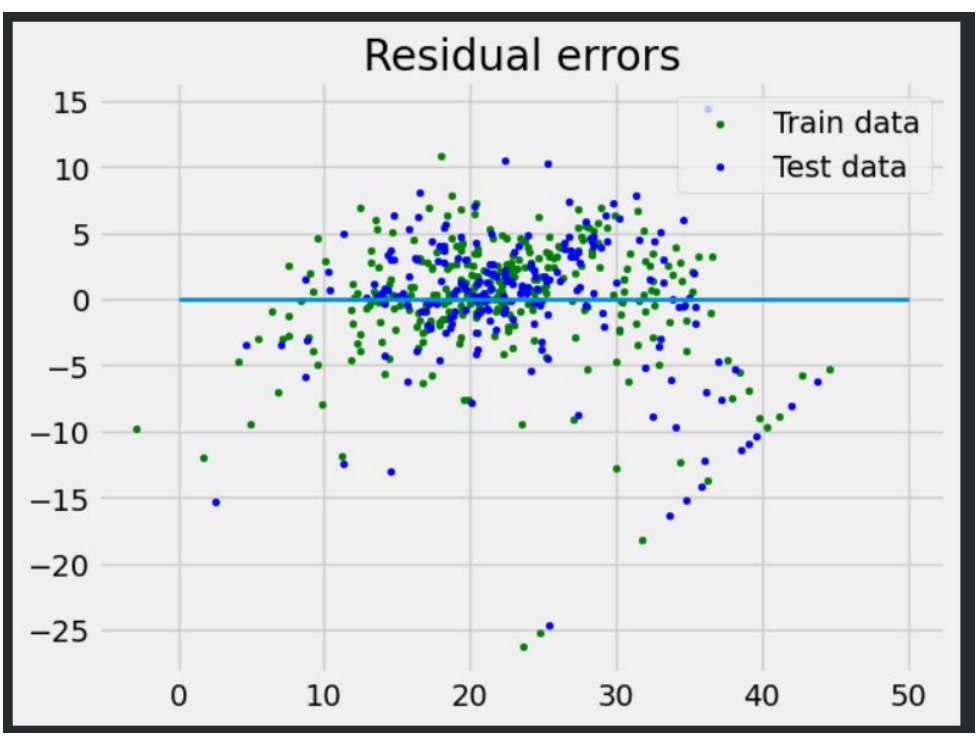
```
from sklearn.model_selection import train_test_split

import matplotlib.pyplot as plt

import numpy as np

from sklearn import datasets, linear_model, metrics

import pandas as pd

data_url = "https://lib.stat.cmu.edu/datasets/boston"

raw_df = pd.read_csv(data_url, sep="\s+",

skiprows=22, header=None)

X = np.hstack([raw_df.values[::2, :],
```

```
raw_df.values[1::2, :2]])

y = raw_df.values[1::2, 2]


X_train, X_test,\

y_train, y_test = train_test_split(X, y,

test_size=0.4,

random_state=1)

reg = linear_model.LinearRegression()

reg.fit(X_train, y_train)


# regression coefficients

print('Coefficients: ', reg.coef_)

# variance score: 1 means perfect prediction

print('Variance score: {}'.format(reg.score(X_test, y_test)))
```

```
Coefficients:  [-8.95714048e-02  6.73132853e-02  5.04649248e-02  2.18579583e+00
 -1.72053975e+01  3.63606995e+00  2.05579939e-03 -1.36602886e+00
  2.89576718e-01 -1.22700072e-02 -8.34881849e-01  9.40360790e-03
 -5.04008320e-01]
Variance score: 0.720905667266174
```

```
# plot for residual error

# setting plot style

plt.style.use('fivethirtyeight')

# plotting residual errors in training data

plt.scatter(reg.predict(X_train),

            reg.predict(X_train) - y_train,

            color="green", s=10,
```

```
        label='Train data')

# plotting residual errors in test data

plt.scatter(reg.predict(X_test),

        reg.predict(X_test) - y_test,

        color="blue", s=10,

        label='Test data')

# plotting line for zero residual error

plt.hlines(y=0, xmin=0, xmax=50, linewidth=2)

# plotting legend

plt.legend(loc='upper right')

# plot title

plt.title("Residual errors")

# method call for showing the plot

plt.show()
```



3. Polynomial Linear Regression

Code:

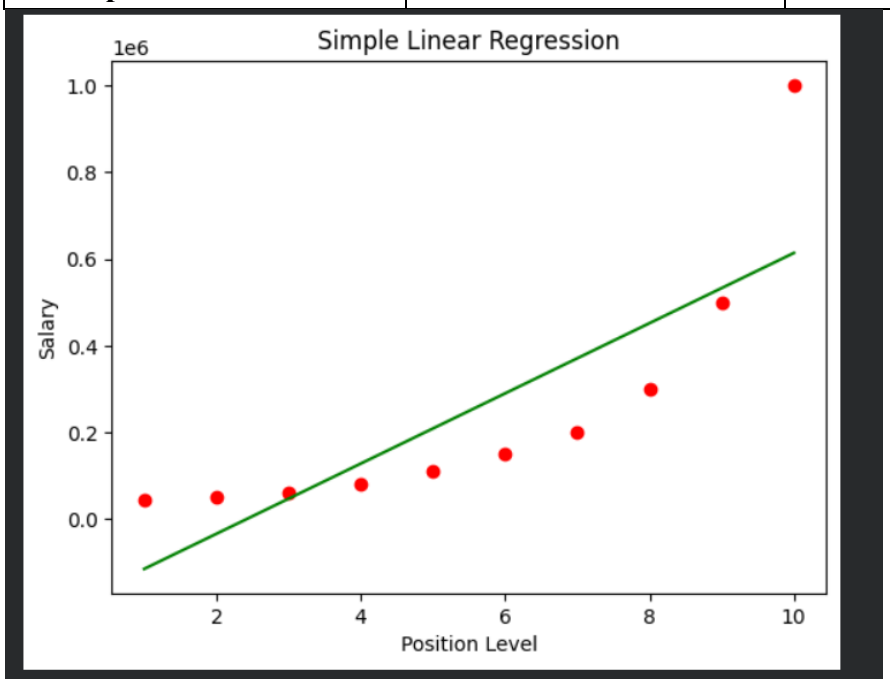
```
import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

import sklearn

import warnings

from sklearn.preprocessing import LabelEncoder

import numpy as np


# Initialize LabelEncoder

le = LabelEncoder()


# Encode the 'Position' column

df['Position_Encoded'] = le.fit_transform(df['Position'])


# Create the desired output array by combining Level, Salary, and encoded Position

desired_output = np.concatenate(

    (X, y.reshape(-1, 1), df[['Position_Encoded']].values),

    axis=1

)

# Display the resulting array

print(desired_output)
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 26/01/2026**	**Enrollment No: 92301733012**

```
•••  [[      1   45000       0]
      [      2   50000       4]
      [      3   60000       8]
      [      4   80000       5]
      [      5  110000       3]
      [      6  150000       7]
      [      7  200000       6]
      [      8  300000       9]
      [      9  500000       1]
      [     10 1000000       2]]
```

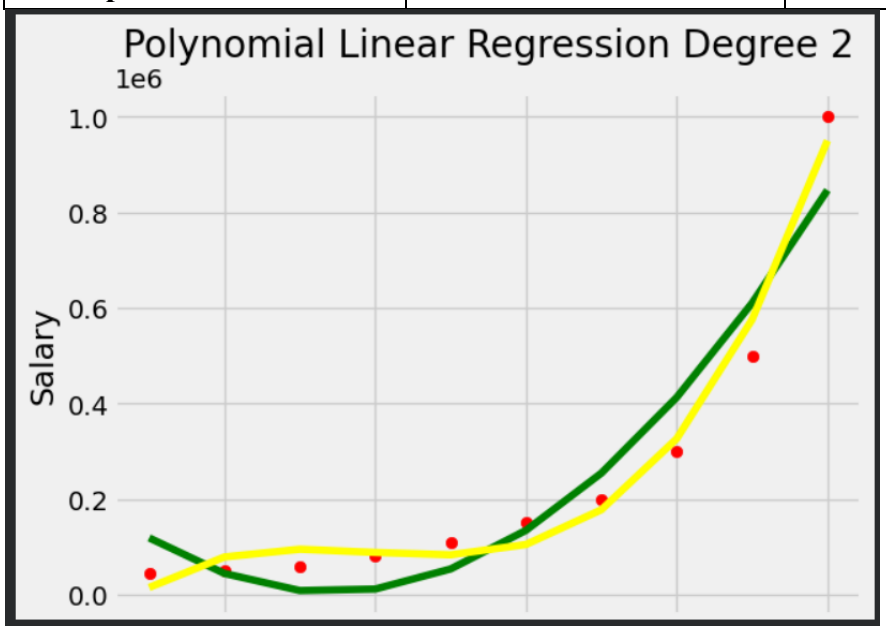
```
from sklearn.linear_model import LinearRegression

lin_reg=LinearRegression()

lin_reg.fit(X,y)

plt.scatter(X,y,color='red')

plt.plot(X,lin_reg.predict(X),color='green')

plt.title('Simple Linear Regression')

plt.xlabel('Position Level')

plt.ylabel('Salary')

plt.show()
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 26/01/2026**	**Enrollment No: 92301733012**



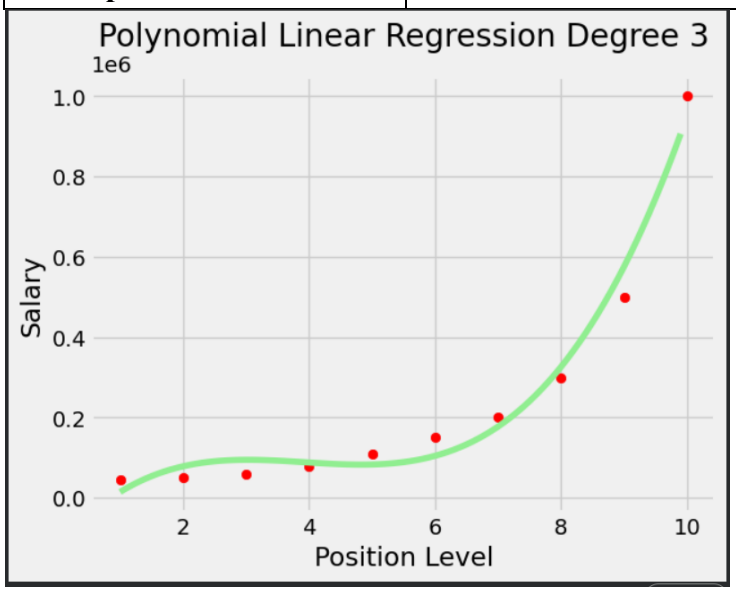

```
plt.style.use('fivethirtyeight')

plt.scatter(X,y,color='red')

plt.plot(X,lin_reg_2.predict(poly_reg2.fit_transform(X)),color='green')

plt.plot(X,lin_reg_3.predict(poly_reg3.fit_transform(X)),color='yellow')

plt.title('Polynomial Linear Regression Degree 2')

plt.xlabel('Position Level')

plt.ylabel('Salary')

plt.show()
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 26/01/2026**	**Enrollment No: 92301733012**




```
plt.style.use('fivethirtyeight')

X_grid=np.arange(min(X),max(X),0.1) # This will give us a vector.We will have to convert this into a matrix

X_grid=X_grid.reshape((len(X_grid),1))

plt.scatter(X,y,color='red')

plt.plot(X_grid,lin_reg_3.predict(poly_reg3.fit_transform(X_grid)),color='lightgreen')


#plt.plot(X,lin_reg_3.predict(poly_reg3.fit_transform(X)),color='green')

plt.title('Polynomial Linear Regression Degree 3')

plt.xlabel('Position Level')

plt.ylabel('Salary')

plt.show()
```

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the best fit line over single feature scattered datapoints using Linear Regression**	
Experiment No: 02	**Date: 26/01/2026**	**Enrollment No: 92301733012**



Observation and Result Analysis:

a. Nature of the dataset

- ✓ The dataset consists of numerical, continuous variables suitable for regression.
- ✓ It includes input feature(s) (X) and a continuous target variable (Y).
- ✓ In the simple linear regression experiment, the dataset contains single-feature scattered datapoints showing an approximately linear relationship.
- ✓ For multiple linear regression, a multivariate dataset (Boston housing dataset) is used with multiple features affecting a continuous output.

b. During Training Process

- ✓ The model initializes parameters (slope and intercept).
- ✓ The hypothesis function predicts output values.
- ✓ The cost function (Mean Squared Error) is calculated to measure prediction error.
- ✓ Batch Gradient Descent iteratively updates parameters to minimize the cost.
- ✓ The loss gradually decreases with iterations, indicating learning.
- ✓ The model searches for the best-fit line that minimizes the overall error.

c. After the training Process

- ✓ The model converges to optimal parameter values.
- ✓ A best-fit regression line is obtained for the dataset.
- ✓ Predictions closely follow the trend of actual data points.

Post Lab Exercise:

a. What are the major assumptions considered in linear regression

- ✓ Linearity

  There is a linear relationship between independent variables and the dependent variable.
- ✓ Independence of Errors

  Residuals (errors) are independent of each other.
- ✓ Homoscedasticity

  The variance of errors is constant for all values of the independent variable.
- ✓ Normality of Errors

  Residuals are normally distributed (important for inference).
- ✓ No Multicollinearity

  Independent variables are not highly correlated with each other.

b. Why MSE is used instead of MAE for calculating the loss function

---

Mean Squared Error (MSE) is preferred over Mean Absolute Error (MAE) because:

1. Differentiable

   MSE is smooth and differentiable, making it suitable for gradient descent optimization.

2. Penalizes Large Errors More

   Squaring the error gives more weight to large errors, helping the model learn better.

3. Mathematical Convenience

   Leads to a closed-form solution (Normal Equation) in linear regression.

c. How can the behaviour of outliers be understood while dealing with the unseen dataset

Outliers can be analyzed and handled using:

1. Residual Analysis

   Large residual values indicate potential outliers.

2. Visualization Techniques
   - Scatter plots
   - Box plots
   - Residual plots
3. Statistical Methods
   - Z-score
   - IQR (Interquartile Range)
4. Model Performance Comparison

   If prediction error increases significantly on unseen data, outliers may be present.

d. Derive the Normal Equation for the Linear Regression.

Let:

- X= feature matrix
- y= target vector
- θ= parameter vector

Hypothesis:

$$h_\theta(X) = X\theta$$

Cost Function (MSE):

$$J(\theta) = \frac{1}{2m}(X\theta - y)^T(X\theta - y)$$

Step 1: Take derivative w.r.t. θ

$$\frac{\partial J(\theta)}{\partial \theta} = \frac{1}{m}X^T(X\theta - y)$$

Step 2: Set derivative to zero

$$X^T(X\theta - y) = 0$$

Step 3: Solve for θ

$$X^T X\theta = X^T y$$

$$\boxed{\theta = (X^T X)^{-1} X^T y}$$